

Hibernate ID Generation Strategies

A practical guide to GenerationType.AUTO, IDENTITY, SEQUENCE, and TABLE — what each one does, when to use it, and how they compare.

In JPA/Hibernate, the **@GeneratedValue** annotation tells Hibernate how the primary key of an entity should be generated. The **strategy** attribute accepts one of four values: **AUTO**, **IDENTITY**, **SEQUENCE**, or **TABLE**. Each one generates IDs in a different way, has different performance characteristics, and fits different databases. This document explains each strategy in the same format: what it is, when to use it, an example, and which databases it suits best — followed by a side-by-side comparison.

1. GenerationType.AUTO

1. What is it?

Hibernate picks the ID generation strategy automatically on your behalf, based on the underlying database dialect being used. Depending on the database, Hibernate will internally choose IDENTITY, SEQUENCE, or TABLE for you. It's the "let Hibernate decide" option.

2. When to use it?

- During early development or prototyping, when you don't want to worry about database-specific details yet.
- When the project might later switch databases and you want Hibernate to adapt automatically.
- For simple applications where fine-grained control over ID generation isn't a priority.

3. Example

```
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String name;
}
```

4. Database

Works with virtually any database (MySQL, PostgreSQL, Oracle, H2, etc.) since Hibernate adapts the real strategy to whatever dialect is configured. Not tied to one database.

2. GenerationType.IDENTITY

1. What is it?

The ID is generated by the database itself using an auto-increment (or identity) column. Hibernate simply inserts the row without the ID and then asks the database for the value it just generated.

2. When to use it?

- When the target database has strong native support for auto-increment columns.
- When the application does not need to know the ID before the INSERT actually happens.
- For simple entities where batching inserts for performance is not a major concern (IDENTITY disables Hibernate's JDBC batch inserts, since each ID is only known after insert).

3. Example

```
@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String email;
}

-- Underlying table (MySQL):
CREATE TABLE customer (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255)
);
```

4. Database

Best suited for databases with native auto-increment support such as MySQL, MariaDB, and SQL Server (IDENTITY columns). Not available in Oracle before 12c.

3. GenerationType.SEQUENCE

1. What is it?

Hibernate uses a database **sequence** object — an independent counter stored in the database — to generate ID values. Hibernate can request a batch of values from the sequence ahead of time, which allows it to know the ID before the INSERT happens.

2. When to use it?

- When the database supports sequences and you want the best performance for bulk inserts.
- When you need the primary key value before persisting the entity (e.g. to use it in related objects before flushing).
- When you want control over sequence behavior, such as increment size, via @SequenceGenerator.

3. Example

```
@Entity
public class Invoice {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
                    generator = "invoice_seq")
    @SequenceGenerator(name = "invoice_seq",
                      sequenceName = "invoice_sequence",
                      allocationSize = 1)
    private Long id;

    private Double amount;
}
```

```
-- Underlying sequence (PostgreSQL):  
CREATE SEQUENCE invoice_sequence START WITH 1 INCREMENT BY 1;
```

4. Database

Ideal for databases that support native sequences, such as PostgreSQL, Oracle, and H2. Not supported by MySQL (before version 8) or SQL Server in the same native way.

4. GenerationType.TABLE

1. What is it?

Hibernate simulates a sequence by using a dedicated database **table** that stores and increments the next available ID value. Every time a new ID is needed, Hibernate reads and updates a row in that table.

2. When to use it?

- When the database supports neither native sequences nor auto-increment columns.
- When you need one ID-generation mechanism that behaves identically across every database (fully database-independent).
- Rarely chosen by preference in modern systems, mainly used as a portable fallback, since it is slower due to extra table reads/writes and potential locking.

3. Example

```
@Entity  
public class Order {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.TABLE,  
                    generator = "order_gen")  
    @TableGenerator(name = "order_gen",  
                    table = "id_generator",  
                    pkColumnName = "gen_name",  
                    valueColumnName = "gen_value",  
                    pkColumnValue = "order_id",  
                    allocationSize = 1)  
    private Long id;  
  
    private String status;  
}  
  
-- Underlying table:  
CREATE TABLE id_generator (  
    gen_name VARCHAR(255) PRIMARY KEY,  
    gen_value BIGINT  
);
```

4. Database

Works on any database (it's just a normal table), so it's used mostly for portability across MySQL, PostgreSQL, Oracle, SQL Server, etc., when a single unified approach is needed.

Comparison

The table below summarizes the four strategies side by side.

Aspect	AUTO	IDENTITY	SEQUENCE	TABLE
Who generates the ID	Hibernate decides (delegates to one of the others)	Database (auto-increment column)	Database (sequence object)	Simulated via a helper table
ID known before INSERT?	Depends on chosen strategy	No	Yes	Yes
Batch insert friendly	Depends	No (breaks JDBC batching)	Yes	Somewhat (extra table I/O)
Performance	Depends on chosen strategy	Fast	Fast, best for bulk ops	Slowest (extra reads/writes, locking)
Portability	High (auto-adapts)	Low (DB must support it)	Medium (DB must support sequences)	Highest (works everywhere)
Typical databases	Any	MySQL, MariaDB, SQL Server	PostgreSQL, Oracle, H2	Any (fallback/portable option)

Key takeaway

There is no single "best" strategy — the right choice depends on the target database and the performance needs of the application. **SEQUENCE** is generally the preferred choice on databases that support it (like PostgreSQL and Oracle) because it offers the best balance of performance and control. **IDENTITY** is a solid, simple choice on databases like MySQL that support auto-increment natively, but it disables batch inserts. **TABLE** is the most portable but the slowest, so it's usually a fallback rather than a first choice. **AUTO** is convenient for quick development since Hibernate picks a strategy for you, but for production systems it's usually better to choose explicitly so you know exactly how your IDs are generated and how that affects performance.